UNIVERSIDADE DE ÉVORA

Universidade de Évora **Docente: Luís Rato**

Relatório Trabalho Sistemas Operativos

Simulador de Sistema Operativo

Engenharia Informática

Tomás Lopes (58147) & David Silva (58637)

20 de junho de 2026

1 Introdução

Este relatório descreve a implementação de um simulador de Sistema Operativo. O objetivo principal do projeto consiste na simulação do ciclo de vida de processos e *threads*, utilizando um modelo de 5 estados (NEW, READY, RUN, BLOCKED e EXIT) e um algoritmo de escalonamento *Round-Robin* com interrupção por limite de tempo. O projeto inclui também um motor de execução para instruções numéricas e um sistema de gestão de memória paginada. Adicionalmente, a simulação garante o rastreamento temporal exato de 100 instantes de execução e assegura o isolamento de variáveis e a correta hierarquia de destruição de *threads* filhas, refletindo com rigor a conversão de programas em formato textual para processamento numérico interno.

2 Arquitetura da Solução

Para garantir a modularidade recomendada, a solução foi dividida em vários componentes independentes:

- **Módulo Principal (`main.c`):** Ponto de entrada do simulador, responsável por inicializar as estruturas do sistema, carregar a configuração inicial e gerir o ciclo temporal.
- **Motor de Execução (`executor.c`):** Responsável por descodificar as instruções lidas da memória e atuar sobre o CPU.
- **Escalonador (`escalonador.c`):** Gere as filas de estados (`Queue`) e aplica as políticas do *Round-Robin* com quantum = 3.
- **Gestor de Memória (`memoria.c`):** Implementa o array linear de inteiros (estritamente limitado a 100 posições) e a alocação de páginas com tamanho fixo de 5 unidades.
- **Tradutor e I/O (`inputs.c` e `output.c`):** Lida com o parsing das instruções, empacotamento numérico de 6 dígitos e exibição do estado do sistema a cada instante.

3 Detalhes de Implementação

3.1 Estruturas de Dados (PCB e TCB)

Para a gestão de contexto, foram criadas as estruturas `Process Control Block` (PCB) e `Thread Control Block` (TCB). A *thread* principal (T00) dita o tempo de vida do processo e é a única a permanecer um instante de tempo no estado NEW. Adicionalmente, a sua passagem para o estado EXIT inicia uma rotina de terminação em cadeia, garantindo a remoção imediata e a libertação de memória de todas as *threads* secundárias do mesmo processo.

3.2 Gestão de Memória e Tradução

A memória foi abstraída num *array* global unidimensional. As variáveis globais (G0-G4) são mapeadas numa página de 5 posições alocada no PCB, enquanto as variáveis locais (L0-L4) ocupam uma página exclusiva no TCB. A distinção entre ambas é assegurada internamente através de um *offset* numérico aplicado durante a leitura. Por fim, as instruções de texto sofrem *parsing* e são empacotadas num código inteiro de 6 dígitos (agrupando *opcode* e argumentos), sendo armazenadas estritamente nas páginas de memória do programa associado. Esta abordagem cumpre o requisito de um processamento interno puramente numérico.

4 Exemplos de Execução

Para validar o simulador, foram utilizados os ficheiros de teste configurados (`00.prog` e `01.prog`). Nesta secção apresentamos dois excertos fundamentais que comprovam o cumprimento dos requisitos estipulados.

O primeiro excerto (Instante 5) demonstra o funcionamento do algoritmo *Round-Robin*. Quando a *thread* P00.T00 esgota o seu limite de tempo (*quantum* = 3), o estado RUN é ocupado pelo `DISPATCHER`, reorganizando a fila READY com as prioridades corretas.

```
5
estado NEW              P03.T00
estado READY            P01.T00          P00.T00
   P02.T00
estado RUN              DISPATCHER
estado BLOCKED
estado EXIT
```

Listing 1: Intervenção do Dispatcher após o esgotamento do tempo

O segundo excerto (Instantes 10 a 12) ilustra a execução da instrução `BLOCK 1` pela *thread* P00.T00. A *thread* sai do processador no instante 10, cumpre o seu castigo de 1 instante no estado BLOCKED durante a intervenção do Dispatcher (instante 11), e regressa imediatamente à fila READY no instante 12.

```
10
estado READY            P02.T00         P03.T00
   P01.T00
estado RUN              P00.T00
estado BLOCKED

11
estado READY            P02.T00         P03.T00
   P01.T00
estado RUN              DISPATCHER
estado BLOCKED          P00.T00

12
estado READY            P03.T00         P01.T00
   P00.T00
estado RUN              P02.T00
estado BLOCKED
```

Listing 2: Transição de estados através da instrução de bloqueio

5 Conclusão

A realização deste trabalho permitiu reforçar e aplicar com sucesso os conceitos aprendidos ao longo do semestre. Foram cumpridos e implementados todos os requisitos estipulados no enunciado, desde o controlo rigoroso das transições de estado e do algoritmo de escalonamento *Round-Robin*, até ao isolamento estrito da memória paginada entre os vários processos e *threads*. Adicionalmente, a adoção de uma arquitetura estruturada e modular revelou-se fundamental para integrar a conversão numérica de instruções com o motor de execução, resultando num simulador robusto, capaz de detetar e gerir com segurança exceções como *Segmentation Fault* e esgotamento da capacidade de memória, sendo totalmente fiel à passagem exata do tempo.